

# Lecture 5: Assembly and Boot

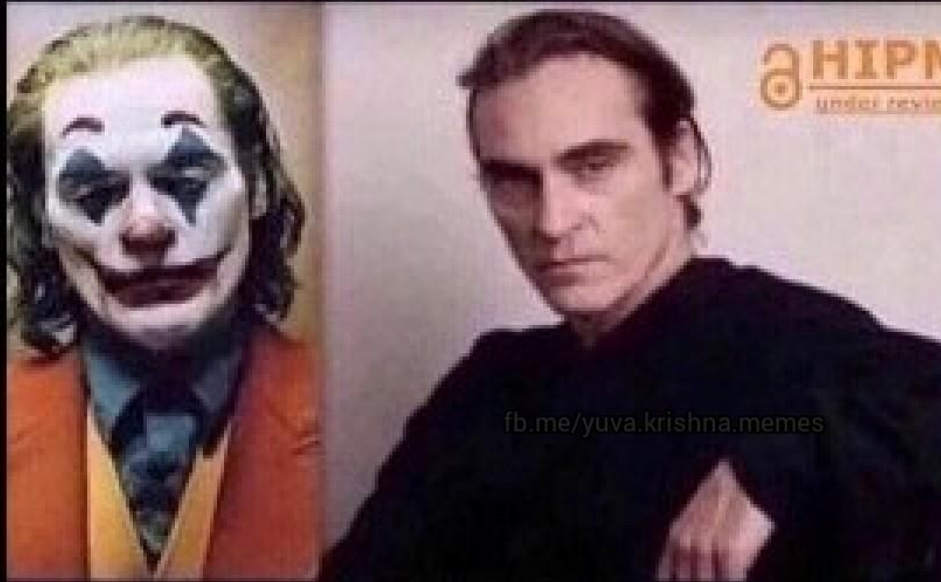
Professor G. Sandoval

05



Some slides adapted by G. Sandoval for CS3224, from slide by Brendan Dolan-Gavitt and Bryant and O'Hallaron, Computer Systems: A programmer's Perspective, Third Edition

# did you know....



in order to play the role of an insane and mentally depressed person in the movie "Joker", Joaquin Phoenix started learning programming in assembly.

# Announcements

- Homework 1 due tonight (with the 24 hrs extension)
- Homework 2 out.
- Recitation on Friday: 4:30 – <https://nyu.zoom.us/j/93463267042>

# Today: Machine Programming Basics

- **Assembly**
  - More examples
  - Function calls
- Booting the OS
  - BIOS
  - Assembly Bootstrap
  - C Bootstrap

# Push/Pop

- Recall: ESP is stack pointer
- On x86 the **stack grows downward** – putting something on the stack decrements the stack pointer
- The **push** instruction **subtracts 4 from ESP**, then **writes to the memory** at address ESP

## Class 5: When we execute: push %eax, what happens to ESP (stack pointer)?

ESP increases by 4

0%

ESP decreases by 4

0%

ESP stays the same

0%

ESP becomes undefined

0%

# Function Calls (call & return)

- Supports creation of functions
- **call <f>**: push the address of next instruction on the stack and jump to f
- Example:
  - 0000 call <f> // pushes 5 and jumps to f
  - 0005 movl %eax, %ebx
- **ret**: pop return address off the stack and jump to it

# Stack Frames

- x86 programs use the stack to support function calls.
- The machine uses the stack to:
  - Pass arguments
  - store return information
  - Save registers
  - Local Storage
- Therefore to use arguments and return information we need to deal with the stack.
- The portion of memory used in the stack for a single procedure call is called a stack frame.
- The stack frame is delimited by: ESP and EBP

# Fill in the blanks

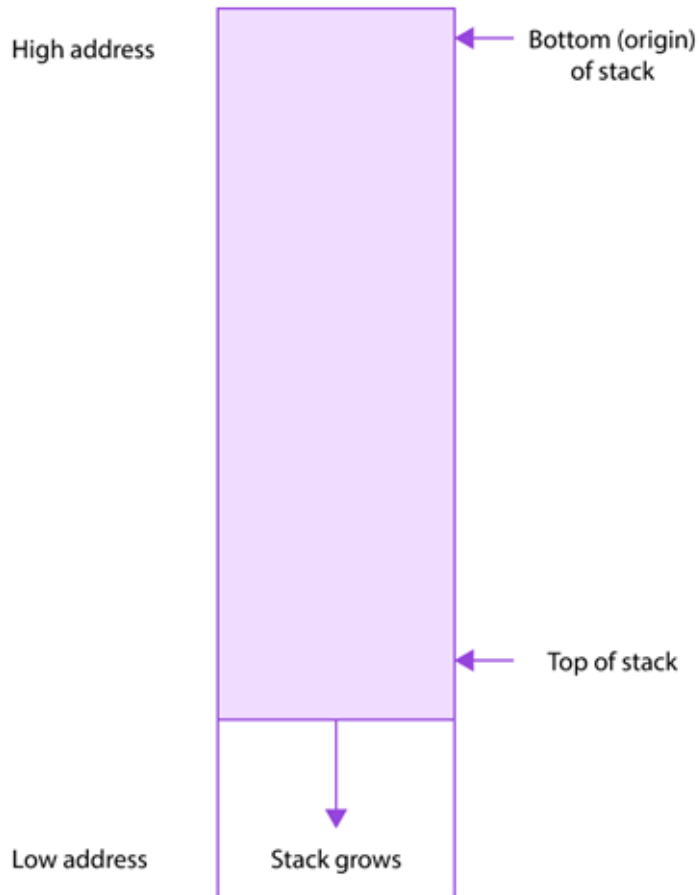
| Example instruction | What it does |
|---------------------|--------------|
| Pushl %eax          |              |
| Popl %eax           |              |
| Call 0x12345        |              |
| Ret                 |              |

# Fill in the blanks

| Example instruction | What it does                        |
|---------------------|-------------------------------------|
| Pushl %eax          | Subl \$4, %esp<br>Movl %eax, (%esp) |
| Popl %eax           | Movl (%esp), %eax<br>Addl \$4, %esp |
| Call 0x12345        | Pushl %eip<br>Movl \$0x12345, %eip  |
| Ret                 | Popl %eip                           |

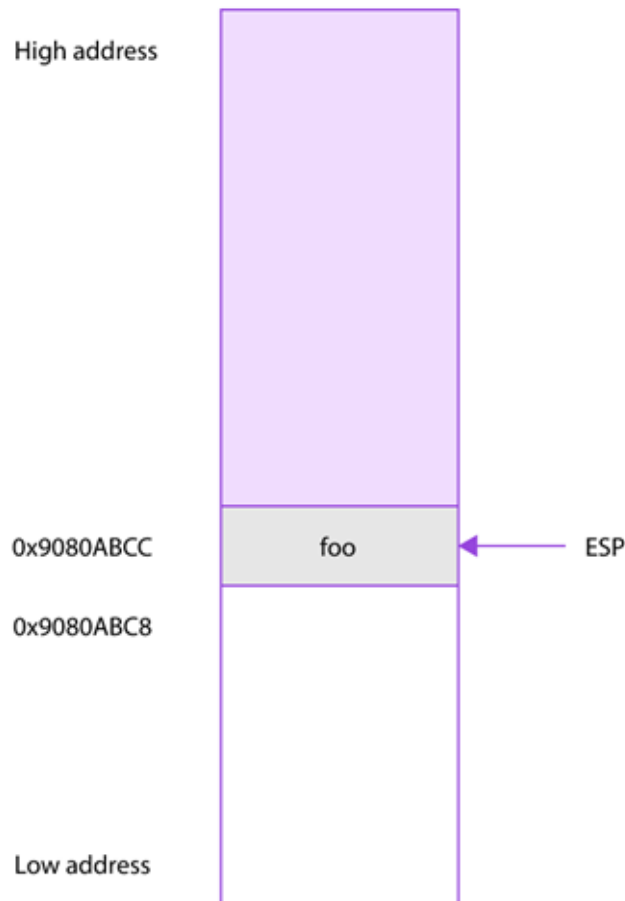
# The stack in x86

- The stack in x86 grows down!
- When we say “top of the stack” on x86 we actually mean the lowest address in the memory area occupied by the stack.
- This might be “unnatural” but keep it in mind.



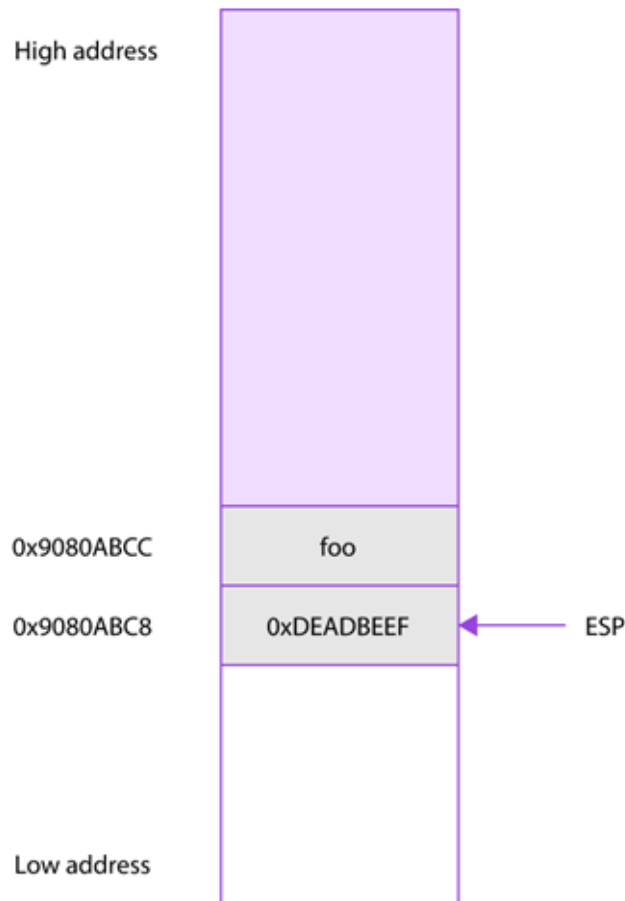
# Pushing and Popping data

- We push data with the stack pointer
- 0x9080ABCC top of stack
- The word at that address has “foo”
- ESP has the address 0x9080ABCC



# Pushing and Popping data

- To push new data we use the **push** instruction.
- push
  - Decrement esp by 4
  - Store its operand in the location esp points to
- Example:
  - Push eax



# Stack Frames

- Inside of a function, we want to access function arguments, as well as have space for local variables.
- Functions typically use `%ebp` register to point to the value `%esp` had at function entry.
- The topmost stack frame is delimited by two register pointers:
  - `%ebp` is the frame pointer (base)
  - `%esp` is the stack pointer
- **So stack arguments can be accessed with `%(ebp)`**

## Class 5: In a function call, which register provides a STABLE reference point for accessing parameters and local variables?

ESP (stack pointer)

0%

EBP (base pointer)

0%

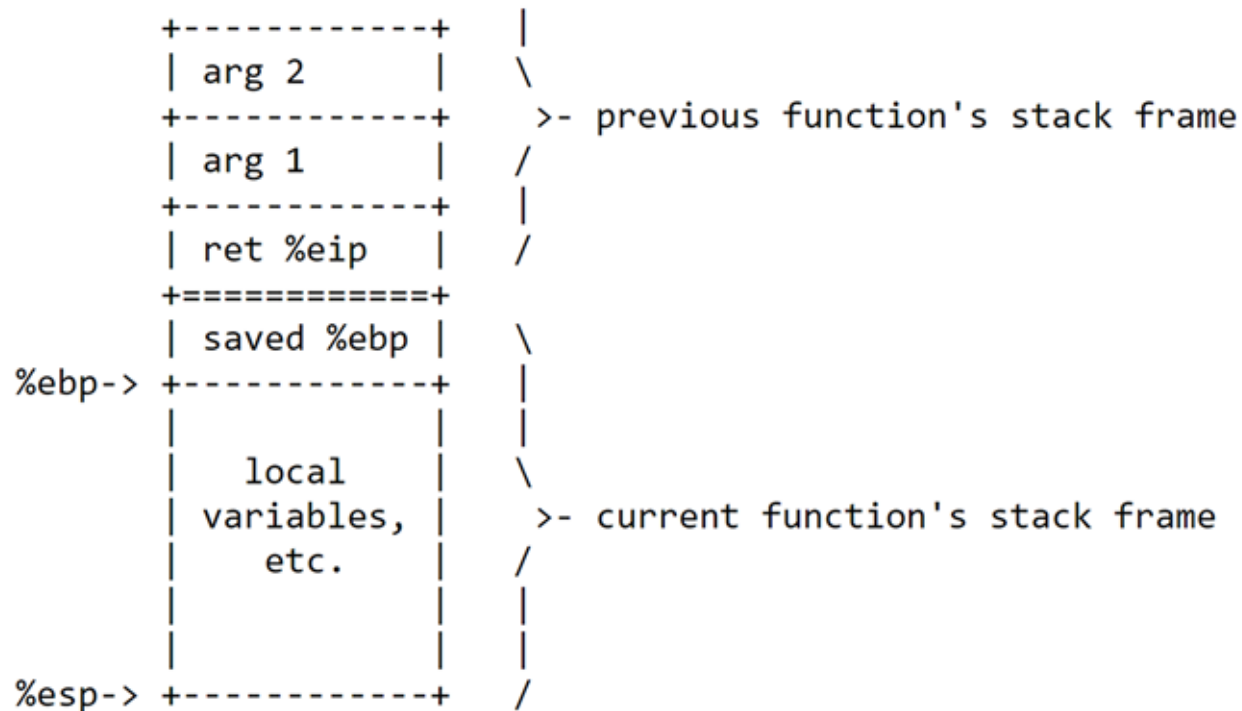
EAX (accumulator)

0%

EIP (instruction pointer)

0%

# Stack Usage



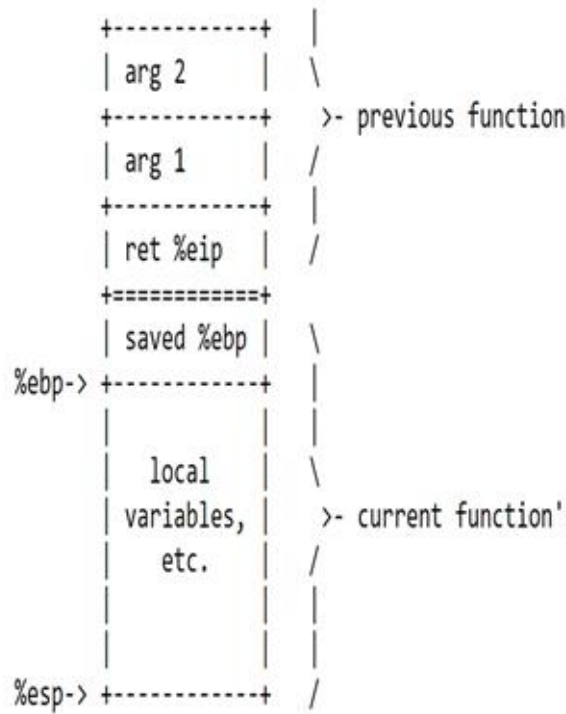
# Function Prologue & Epilogue

- The need to set up a stack frame gives rise to a common function prologue:

```
push %ebp           // save old value of ebp
mov %esp, %ebp      // set base pointer
sub $X, %esp        // reserve space for locals
```

- At the end, we can clean up with just the epilogue

```
mov %ebp, %esp      // restore stack pointer
pop %ebp            // restore base pointer
ret                 // return pops return address
```



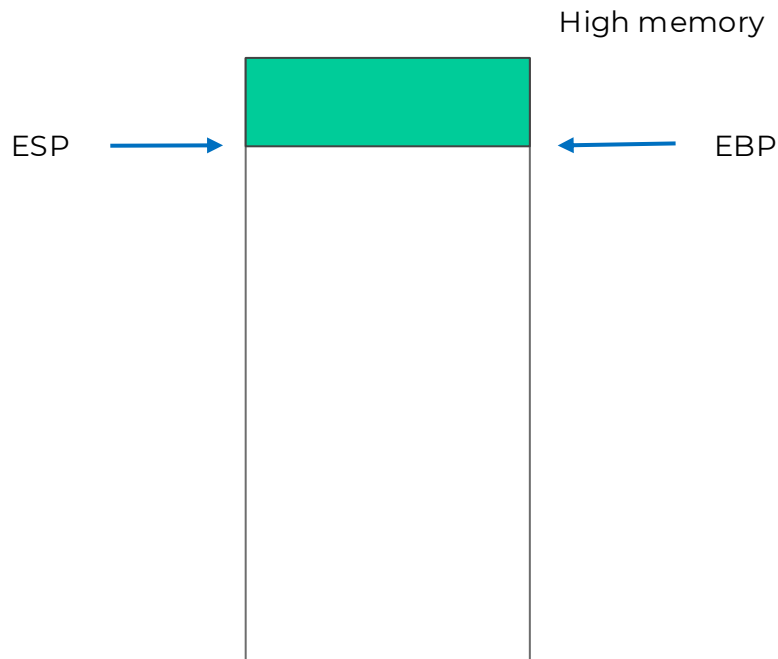
# Aside: Buffer Overflow

- Consider a function like:  

```
int f(int arg1, int arg2) {  
    char x[4];  
    ...  
}
```
- Stack frame for function shown at right
- Suppose inside f, we write past the end of array x?
- What gets overwritten? What happens when we execute ret?

|                    |
|--------------------|
| Arg2               |
| Arg1               |
| Return address     |
| Saved base pointer |
| X[3]               |
| X[2]               |
| X[1]               |
| X[0]               |

# Example:



```
int AddMe (int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main()
{
    AddMe (10, 20);
    print...
```

## Class 5: In a function call, which register provides a STABLE reference point for accessing parameters and local variables?

ESP (stack pointer)

0%

EBP (base pointer)

0%

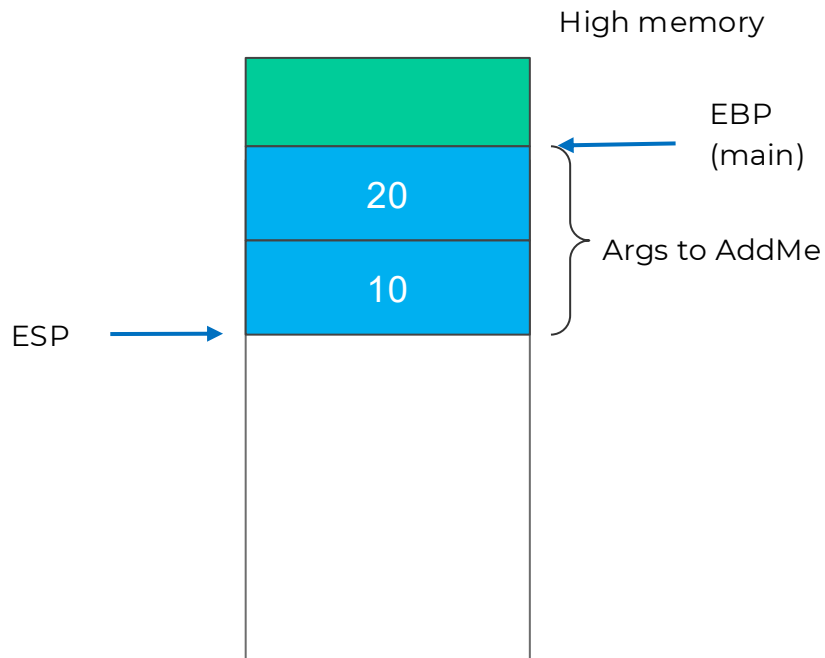
EAX (accumulator)

0%

EIP (instruction pointer)

0%

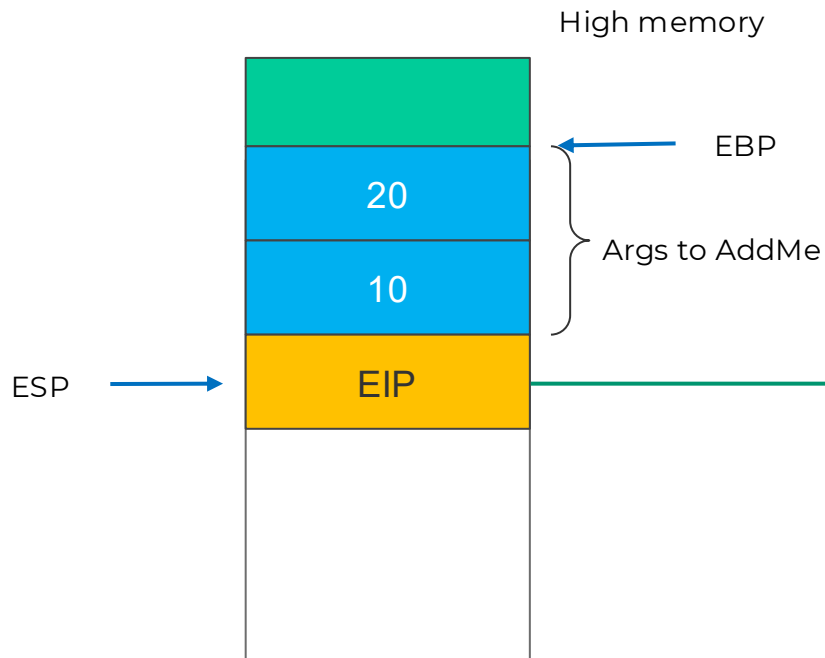
# Walking the Call Stack



```
int AddMe (int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main()
{
    AddMe (10, 20);
    print...
```

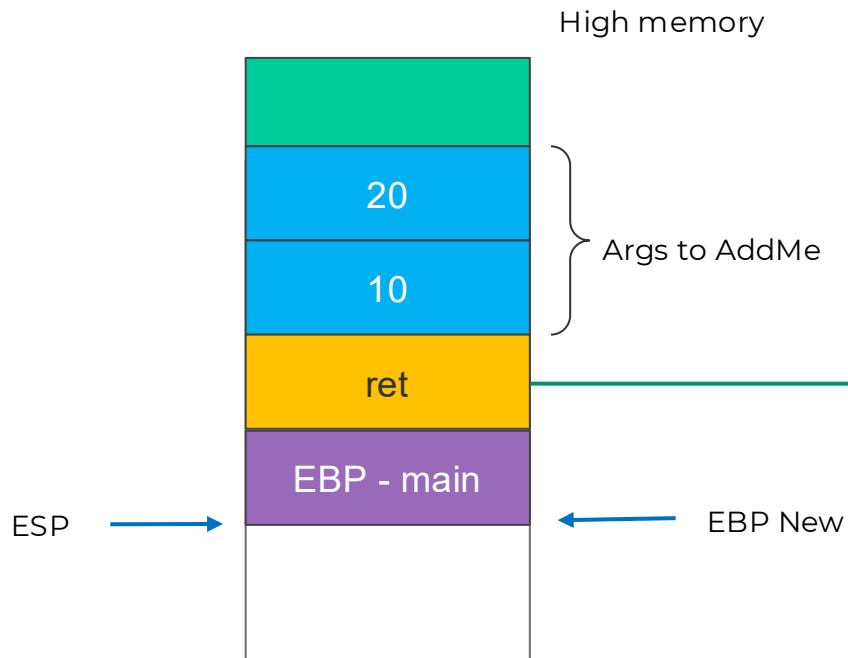
# Walking the Call Stack



```
int AddMe (int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main ()
{
    AddMe (10, 20);
    print...
```

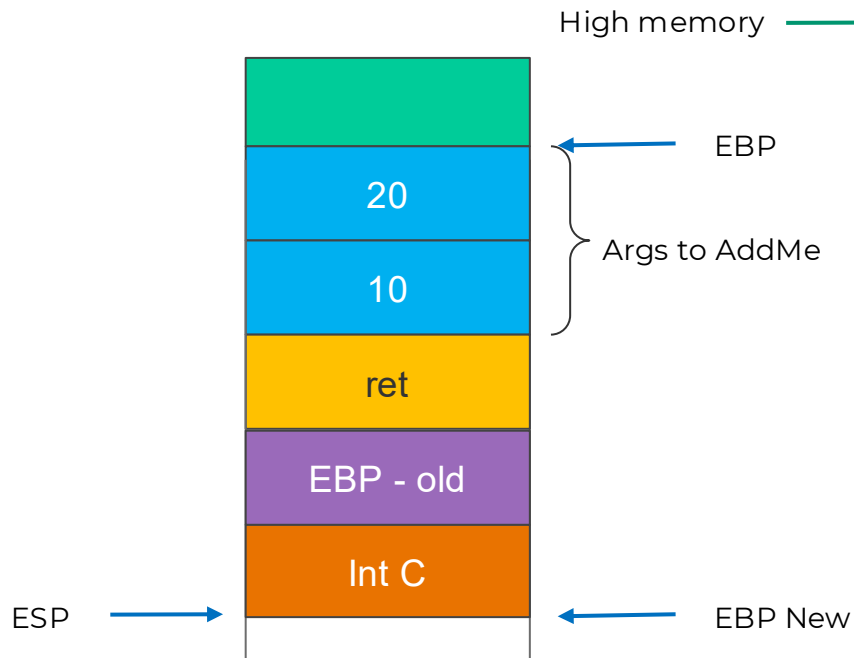
# Walking the Call Stack



```
int AddMe (int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main()
{
    AddMe (10, 20);
    print...
```

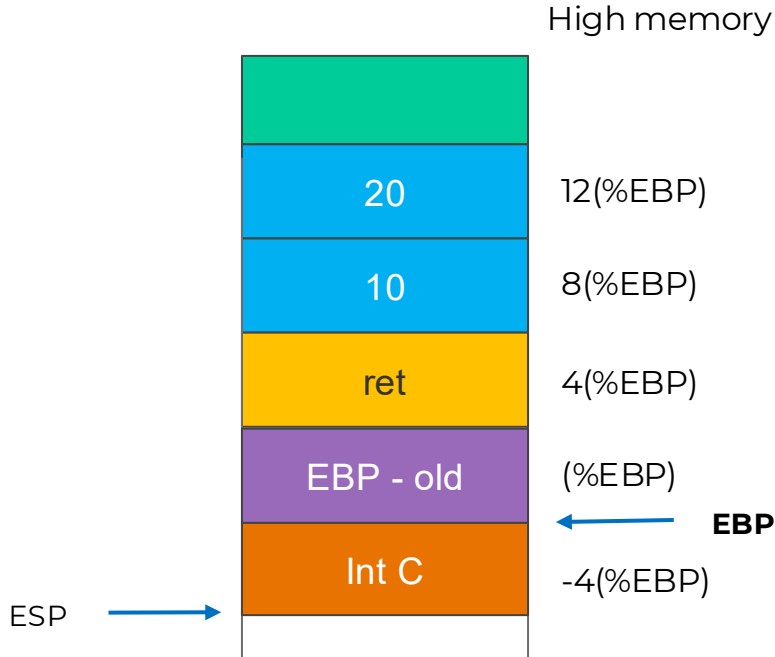
# Walking the Call Stack



```
int AddMe (int a, int b)
{
    int c;
    c = a + b;
    return c;
}

main ()
{
    AddMe (10, 20);
    print...
```

# To reference data on the stack



# Stack Usage

- **GCC dictates how the stack is used. Contract between caller and callee on x86**
- **Entry to a function: (just after call)**
  - %eip points at the **first instruction** of the function
  - %ebp points to the **return address**
  - %ebp+4 points at **first argument** to function
- **After ret instruction**
  - %eip contains return address
  - %esp points at argument pushed by caller
  - Called function may have trashed arguments
  - %eax contains return value (or thrash if function is void)
  - %eax, %ecx and %edx may be trashed

# Calling Conventions

- A calling convention is a system-defined convention that tells you how to call a function
- In general a **calling convention** will:
  - Say how to pass arguments to the function
  - Say who is responsible for cleaning up arguments
  - Specify where the return value will be placed
  - Say which registers will be preserved by the callee and which might be trashed

## Class 5: Who should be responsible for cleaning up function arguments from the stack?

The caller (function that made the call)

0%

The callee (function that was called)

0%

The operating system

0%

It doesn't matter

0%

# Gcc (cdecl) Calling Convention (32-bit)

- **Caller** is responsible for **cleaning arguments** off the stack.
- This allows functions with variable numbers of arguments (printf)
- %eax %ecx, %edx may be trashed by callee (must be saved by **caller**)
- %ebp, %ebx, %esi, %edi must be preserved by **callee**

```
; Caller  
push arg2  
push arg1
```

```
call Function
```

```
add esp, 8 ; Cleanup for 2  
arguments, assuming  
each is 4 bytes
```

# Stdcall

- Often used in Windows API Functions
- Callee Cleans up
- Will adjust the stack pointer before returning to the caller
- Example

; Callee

ret 8 ; Cleanup for 2 arguments, assuming each is 4 bytes

# Walkthrough

- A great way to understand what's going on is to step through a program in **gdb**
- Or the VSCode **debugger**
- You'll also use a debugger to debug the OS, so it's good to become familiar with it
- Another option (at least in assembly):
- <http://emu86.pythonanywhere.com>

# Today: Machine Programming Basics

- Assembly
  - More examples
  - Function calls
- GDB
- Booting the OS
  - **BIOS**
  - Assembly Bootstrap
  - C Bootstrap

## Class 5: When you press the power button on a computer, what runs FIRST?

The OS kernel

0%

The bootloader

0%

BIOS/UEFI firmware

0%

Device Drivers

0%

# BIOS

- When an x86 PC boots, it starts executing the **BIOS**.
- **BIOS**
  - Small bit of code stored on the motherboard
  - Prepares the hardware
  - Transfers control to the **OS** (Bootsector)
- **BootSector:**
  - First 512 bytes of the boot disk.
  - Contains the boot loader

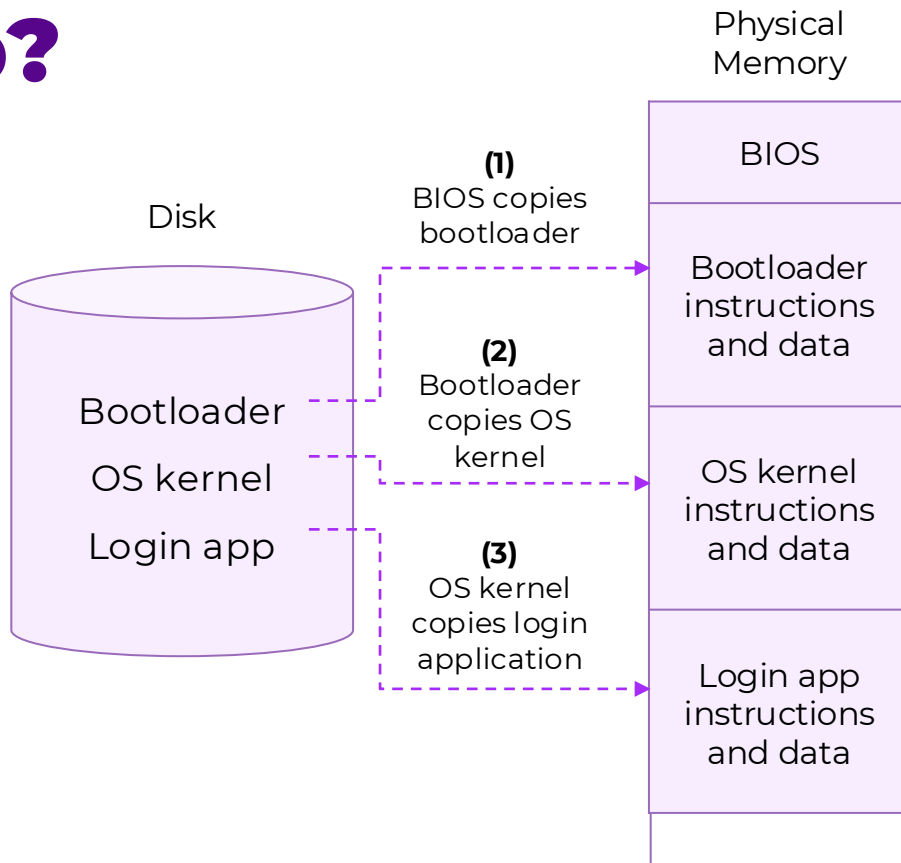
# xv6 Bootloader

- Note: x86 processors start out in 16-bit *real mode* (pretends to be an 8088 processor from 1979)
- **BIOS** loads the bootloader (512 bytes) at 0x7c00
- **Bootloader's** job:
  - Put the processor in a more modern operating mode(386 from 8088),
  - Load the xv6 **kernel** from disk into memory,
  - Transfer **control** to the **kernel**.

- The **xv6 boot loader** comprises two parts:
  - **Assembly Bootstrap (bootasm.S)**: written in a combination of 16-bit and 32-bit x86 assembly
  - **C Bootstrap (bootmain.c)**: written in C

# How to bootstrap?

- Power up a computer
- Processor resets
  - Set to known state
  - Jump to ROM code (for x86 PC, this is **BIOS**)
- Load in the boot loader from stable storage
- Jump to the boot loader
- Load the rest of the operating system
- Initialize and run



# X86 operating modes

## Real Mode

- **-Default on reset/boot** (BIOS starts here).
- **16-bit mode** (20-bit addresses → max 1 MB addressable memory).
- No protection, no privilege levels, no virtual memory.
- Direct segment:offset addressing (segment \* 16 + offset).
- Used today only for **bootstrapping**.

## Protected Mode

- Introduced to support multitasking OSes.
- **32-bit (on 386+)** with flat 4 GB address space.
- Adds:
- **Segmentation with privilege levels (rings 0–3).**
- **Virtual memory** via paging.
- Hardware **protection** (can't access kernel memory from user mode).
- Still the **main workhorse mode** for many 32-bit OSes (e.g., Windows XP).

# X86 operating modes (2)

## Long Mode (x86-64, AMD64, Intel 64)

- Used in all modern 64-bit CPUs.
- Requires setting special MSRs and enabling paging.
- Has two sub-modes:
- **64-bit mode:** 64-bit registers, 64-bit virtual addresses (typically 48-bit usable).
- **Compatibility mode:** Run 16-bit or 32-bit protected-mode apps under a 64-bit OS.
- Segmentation mostly disabled; paging is mandatory.

## System Management Mode (SMM)

- Special-purpose mode for firmware (BIOS/UEFI, ACPI).
- Entered via **the System Management Interrupt (SMI)**.
- OS cannot disable or directly see it.
- Used for power management and hardware control.

```

asm bootasm.S
1  #include "asm.h"
2  #include "memlayout.h"
3  #include "mmu.h"
4
5  # Start the first CPU: switch to 32-bit protected mode, jump into C.
6  # The BIOS loads this code from the first sector of the hard disk into
7  # memory at physical address 0x7c00 and starts executing in real mode
8  # with %cs=0 %ip=7c00.
9
10 .code16                # Assemble for 16-bit mode
11 .globl start
12 start:
13     cli                # BIOS enabled interrupts; disable
14
15     # Zero data segment registers DS, ES, and SS.
16     xorw    %ax,%ax     # Set %ax to zero
17     movw    %ax,%ds     # -> Data Segment
18     movw    %ax,%es     # -> Extra Segment
19     movw    %ax,%ss     # -> Stack Segment
20
21     # Physical address line A20 is tied to zero so that the first PCs
22     # with 2 MB would run software that assumed 1 MB. Undo that.
23 seta20.1:
24     inb     $0x64,%al    # Wait for not busy
25     testb   $0x2,%al
26     jnz     seta20.1
27
28     movb     $0xd1,%al    # 0xd1 -> port 0x64
29     outb     %al,$0x64
30
31 seta20.2:
32     inb     $0x64,%al    # Wait for not busy
33     testb   $0x2,%al
34     jnz     seta20.2
35

```

```

36     movb     $0xdf,%al    # 0xdf -> port 0x60
37     outb     %al,$0x60
38
39     # Switch from real to protected mode. Use a bootstrap GDT that makes
40     # virtual addresses map directly to physical addresses so that the
41     # effective memory map doesn't change during the transition.
42     lgdt     gdt_desc
43     movl     %cr0,%eax
44     orl      $CR0_PE,%eax
45     movl     %eax,%cr0
46
47
48     # Complete transition to 32-bit protected mode by using long jmp
49     # to reload %cs and %eip. The segment descriptors are set up with no
50     # translation, so that the mapping is still the identity mapping.
51     ljmp     $(SEG_KCODE<<3), $start32
52
53 .code32 # Tell assembler to generate 32-bit code now.
54 start32:
55     # Set up the protected-mode data segment registers
56     movw     $(SEG_KDATA<<3), %ax # Our data segment selector
57     movw     %ax,%ds             # -> DS: Data Segment
58     movw     %ax,%es             # -> ES: Extra Segment
59     movw     %ax,%ss             # -> SS: Stack Segment
60     movw     $0,%ax              # Zero segments not ready for use
61     movw     %ax,%fs             # -> FS
62     movw     %ax,%gs             # -> GS
63
64     # Set up the stack pointer and call into C.
65     movl     $start,%esp
66     call     bootmain

```

# 1. Initial Setup

- **During the BIOS's hardware initialization, it enabled hardware interrupts**
  - We don't know what its interrupt handlers look like, and we don't want to deal with interrupts yet
  - Use **cli** instruction to disable interrupts.
- **We also don't know the state of segment registers, so we clear them.**

# 1. Initial Setup

```
10 .code16                      # Assemble for 16-bit mode
11 .globl start
12 start:
13     cli                      # BIOS enabled interrupts; disable
14
15     # Zero data segment registers DS, ES, and SS.
16     xorw    %ax,%ax          # Set %ax to zero
17     movw    %ax,%ds          # -> Data Segment
18     movw    %ax,%es          # -> Extra Segment
19     movw    %ax,%ss          # -> Stack Segment
20
```

Clear Interrupts

Clear Segments





# The A20 Line

## What is it?

- On the original 8086, only **20 address lines** → could access up to **1 MB (0xFFFFF)**.
- Any address above 1 MB “wrapped around” to the beginning

## The problem with the 80286+

2. New CPUs supported more memory (>1 MB).
3. But some old DOS programs relied on wraparound.
4. Solution: add a gate on **Address Line 20 (A20)**.

## Two cases

- **A20 Disabled:** 0x100000 wraps back to 0x00000.
- **A20 Enabled:** 0x100000 goes to extended memory.

## Why do we care in the bootloader?

- Before entering **protected mode**, we must enable the A20 line.
- Otherwise, the kernel might overwrite low memory instead of using extended memory.

# Wraparound

## **A20 Disabled:**

Addresses above 1MB wrap around.  
Addresses from 0x00000 – 0xFFFFF (1 MB)

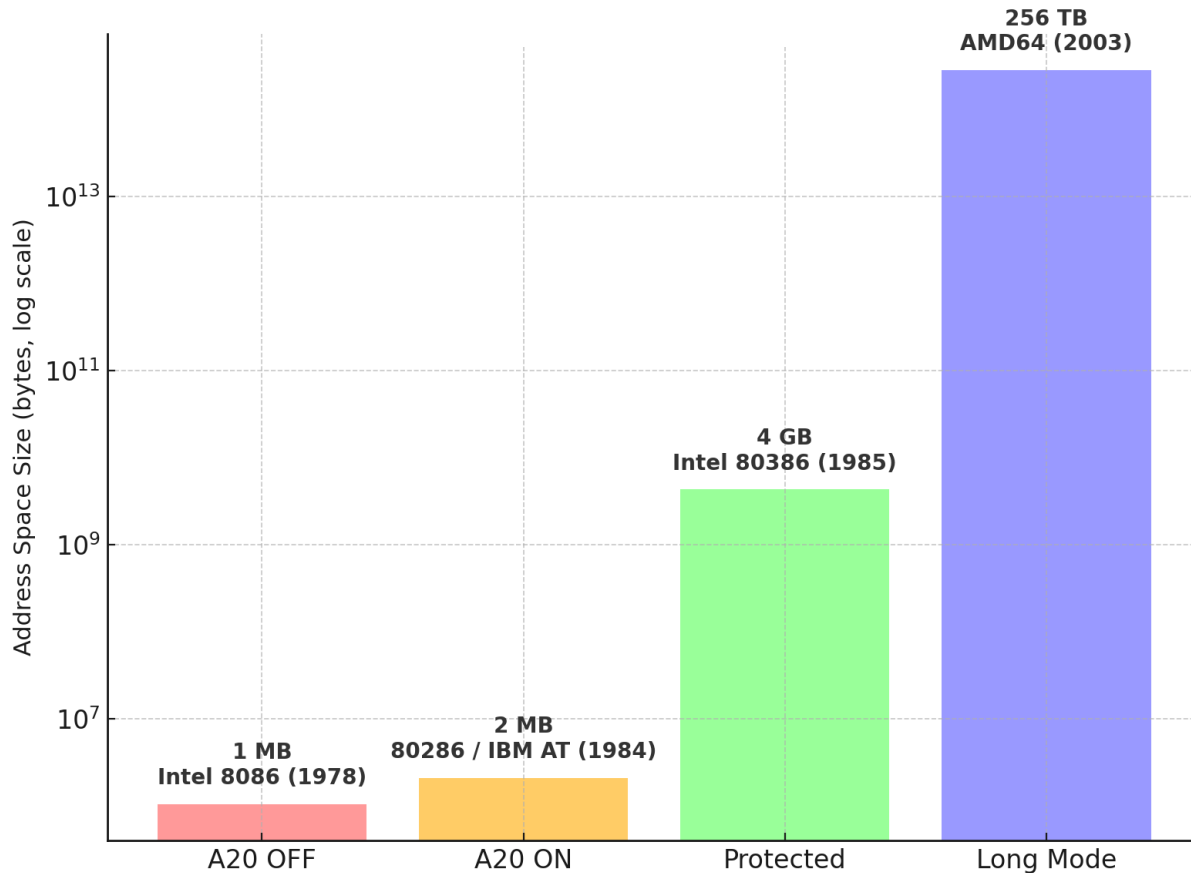
`0x100000 -> 0x00000`

## **A20 Enabled:**

Addresses above 1MB go to extended memory.  
Addresses from 0x00000 – 0x1FFFFFF (2MB)

`0x100000 -> 0x100000`

# X86 Address Spaces by Mode



## 2. Enable Line 20

```
21 | # Physical address line A20 is tied to zero so that the first PCs
22 | # with 2 MB would run software that assumed 1 MB. Undo that.
23 | seta20.1:
24 |     inb    $0x64,%al          # Wait for not busy
25 |     testb  $0x2,%al
26 |     jnz    seta20.1
27 |
28 |     movb   $0xd1,%al          # 0xd1 -> port 0x64
29 |     outb   %al,$0x64
30 |
31 | seta20.2:
32 |     inb    $0x64,%al          # Wait for not busy
33 |     testb  $0x2,%al
34 |     jnz    seta20.2
35 |
```

# 3. Protected Mode setup

- To run **32-bit code**, we need to transition into *protected mode*
- A key feature here is that the **meaning of the segment registers** (%cs, %ds, etc.) **changes**
- Instead of just holding extra bits for memory addresses, they *select* a particular *segment descriptor*
  - *Segment descriptor* gives the **base address, size**, and **permissions** of that memory segment

# Segments in xv6

```
39  # Switch from real to protected mode. Use a bootstrap GDT that makes
40  # virtual addresses map directly to physical addresses so that the
41  # effective memory map doesn't change during the transition.
42  lgdt    gdtdesc
43  movl    %cr0, %eax
44  orl     $CR0_PE, %eax
45  movl    %eax, %cr0
```

- As with most modern OSes, xv6 makes **very little use** of segmentation
- The segments it defines just give a 1-1 mapping between logical and physical addresses
- Later, it will set up the **paging hardware** and use **virtual addresses**

## 4. Jump to 32 bit mode

- Finally we can jump into **protected mode**. The change happens only once a segment register is modified, so we execute a *long jump* to set %cs
- From here on out, we're executing 32-bit x86 code

```
48  # Complete transition to 32-bit protected mode by using long jmp
49  # to reload %cs and %eip. The segment descriptors are set up with no
50  # translation, so that the mapping is still the identity mapping.
51  ljmp    $(SEG_KCODE<<3), $start32
52
```

## 5. Protected Mode Initialization

```
53  .code32 # Tell assembler to generate 32-bit code now.
54  start32:
55      # Set up the protected-mode data segment registers
56      movw    $(SEG_KDATA<<3), %ax    # Our data segment selector
57      movw    %ax, %ds                # -> DS: Data Segment
58      movw    %ax, %es                # -> ES: Extra Segment
59      movw    %ax, %ss                # -> SS: Stack Segment
60      movw    $0, %ax                 # Zero segments not ready for use
61      movw    %ax, %fs                # -> FS
62      movw    %ax, %gs                # -> GS
63
```

## 6. Onward to C

- Finally, we can set up the stack and jump into C code
- The stack pointer is set to **0x7c00** – the address of the start of the **bootloader**

```
64  | # Set up the stack pointer and call into C.  
65  | movl    $start, %esp  
66  | call    bootmain
```

# Note:

- The boot loader has “nursed” the processor
  - Through an evolution from **8088 (16 bit addresses)**
  - Through **80286 (20 bit addresses)**
  - To **80386 (32 bit addresses)**

# bootmain.c

- Reads in the kernel header from the hard disk
- Parses the kernel headers – it's a standard ELF (Executable and Linkable Format) file that gcc emits
- Reads each section (kernel code, data) into memory at the address specified in the ELF headers
- Finally, calls the **entry point** defined in the ELF file

# Bootmain.c

```
17 void bootmain(void) {
18     struct elfhdr* elf;
19     struct proghdr *ph, *eph;
20     void (*entry)(void);
21     uchar* pa;
22
23     elf = (struct elfhdr*) 0x10000; // scratch space
24
25     // Read 1st page off disk
26     readseg((uchar*) elf, 4096, 0);
27
28     // Is this an ELF executable?
29     if(elf->magic != ELF_MAGIC)
30         return; // let bootasm.S handle error
31
32     // Load each program segment (ignores ph flags).
33     ph = (struct proghdr*) ((uchar*) elf + elf->phoff);
34     eph = ph + elf->phnum;
35     for(; ph < eph; ph++) {
36         pa = (uchar*) ph->paddr;
37         readseg(pa, ph->filesz, ph->off);
38         if(ph->memsz > ph->filesz)
39             stosb(pa + ph->filesz, 0, ph->memsz - ph->filesz);
40     }
41
42     // Call the entry point from the ELF header.
43     // Does not return!
44     entry = (void (*)(void))(elf->entry);
45     entry();
46 }
```

# kernel.ld

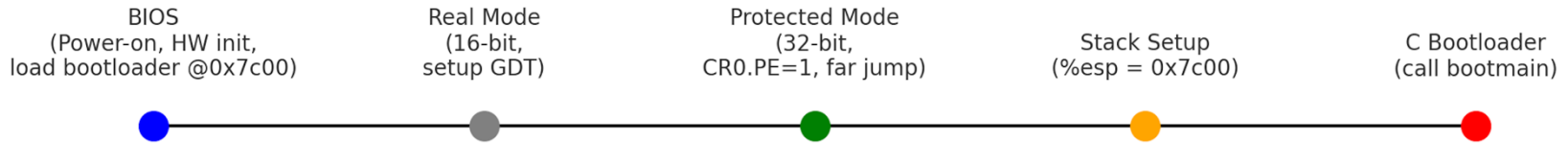
- The kernel will eventually be loaded at **0x80100000** (virtual)
- For now though, we have only **physical memory**, not virtual memory
- Luckily, we can use a *linker* script to specify the physical load address, along with the entry point

# Kernel Entry Point

- Defined in **entry.S**
- This is where we finally turn on paging, but for now we just use a very simple mapping:

0x80000000 => 0x00000000

# Timeline: From Power-On to C Bootloader



# Kernel Main

```
// Bootstrap processor starts running C code here.
// Allocate a real stack and switch to it, first
// doing some setup required for memory allocator to work.
int
main(void)
{
    kinit1(end, P2V(4*1024*1024)); // phys page allocator
    kvmalloc(); // kernel page table
    mpinit(); // collect info about this machine
    lapicinit();
    seginit(); // set up segments
    cprintf("\ncpu%d: starting xv6\n\n", cpu->id);
    picinit(); // interrupt controller
    ioapicinit(); // another interrupt controller
    consoleinit(); // I/O devices & their interrupts
    uartinit(); // serial port
    pinit(); // process table
    tvinit(); // trap vectors
    binit(); // buffer cache
    fileinit(); // file table
    ideinit(); // disk
    if(!ismp)
        timerinit(); // uniprocessor timer
    startothers(); // start other processors
    kinit2(P2V(4*1024*1024), P2V(PHYSTOP)); // must come after startothers()
    userinit(); // first user process
    // Finish setting up this processor in mpmain.
    mpmain();
}
```

# Creating the First Process

- This happens in `userinit()` – after we've initialized lots of other hardware  
[proc.c:79]
- The first process will run the code in `initcode.S`
- To do that it needs:
  - A process structure filled out and assigned a slot in the process table (`allocproc()`)
  - An address space (`setupkvm()`)

# initcode.S

```
# exec(init, argv)
.globl start
start:
    pushl $argv
    pushl $init
    pushl $0 // where caller pc would be
    movl $SYS_exec, %eax
    int $T_SYSCALL
[...]
```

```
# char init[] = "/init\0";
init:
    .string "/init\0"
```